A CAMADA DE PROGRAMAÇÃO

6 - Linguagens de Programação de Baixo Nível e Pseudocódigo

O Capítulo 6 é o primeiro capítulo da camada de programação. Nos Capítulos 2 e 3, abordamos as informações básicas necessárias para a compreensão de um sistema computacional, incluindo sistemas numéricos e maneiras de representar diferentes tipos de informação em um computador. Nos Capítulos 4 e 5, abordamos os componentes de hardware de um computador. Agora, a ênfase muda do que é um sistema computacional para como usá-lo.

Iniciamos este capítulo examinando o código de máquina, a linguagem de programação de nível mais baixo de todas – a linguagem embutida na máquina. Em seguida, avançamos um nível para a linguagem assembly, uma linguagem na qual podemos usar uma combinação de letras para representar uma instrução em linguagem de máquina. Por fim, introduzimos o conceito de pseudocódigo como uma forma de expressar algoritmos.

OBJETIVOS

Após estudar este capítulo, você deverá ser capaz de:

- listar as operações que um computador pode realizar.
- descrever os recursos importantes da máquina virtual Pep/8.
- distinguir entre o modo de endereçamento imediato e o modo de endereçamento direto.
- escrever um programa simples em linguagem de máquina.
- distinguir entre linguagem de máquina e linguagem assembly.
- descrever as etapas de criação e execução de um programa em linguagem assembly.
- escrever um programa simples em linguagem assembly.
- distinguir entre instruções para o montador e instruções a serem traduzidas.
- distinguir entre seguir um algoritmo e desenvolver um.
- descrever as construções de pseudocódigo usadas para expressar um algoritmo.
- usar pseudocódigo para expressar um algoritmo.
- descrever duas abordagens de teste.
- projetar e implementar um plano de teste para um programa simples em linguagem assembly.

6.1 Operações de Computador

As linguagens de programação que usamos devem espelhar os tipos de operações que um computador pode realizar. Portanto, vamos começar nossa discussão repetindo a definição de um computador: Um computador é um dispositivo eletrônico programável que pode armazenar, recuperar e processar dados.

As palavras operacionais aqui são programável, armazenar, recuperar e processar. Em um capítulo anterior, destacamos a importância de compreender que dados e instruções para manipular os dados são logicamente os mesmos e podem ser armazenados no mesmo local. É isso que a palavra programável significa neste contexto. As instruções que manipulam dados são armazenadas na máquina junto com os dados. Para alterar o que o computador faz com os dados, alteramos as instruções.

Armazenar, recuperar e processar são ações que o computador pode executar com os dados. Ou seja, as instruções que a unidade de controle executa podem armazenar dados na memória da máquina, recuperar dados da memória da máquina e processar os dados de alguma forma na unidade aritmética/lógica. A palavra processar é muito geral. No nível da máquina, processar envolve a execução de operações aritméticas e lógicas em valores de dados.

De onde vêm os dados armazenados na memória do computador? Como o ser humano consegue ver o que está armazenado lá, como os resultados de algum cálculo? Existem outras instruções que especificam a interação entre um dispositivo de entrada e a CPU e entre a CPU e um dispositivo de saída.

6.2 Linguagem de Máquina

Como apontamos no Capítulo 1, as únicas instruções de programação que um computador realmente executa são aquelas escritas em linguagem de máquina, as instruções incorporadas ao hardware de um computador específico. Inicialmente, os humanos não tinham escolha a não ser escrever programas em linguagem de máquina, pois outras linguagens de programação ainda não haviam sido inventadas.

* Linguagem de máquina: a linguagem composta por instruções codificadas em binário que é usada diretamente pelo computador.

Então, como as instruções do computador são representadas? Lembre-se de que cada tipo de processador tem seu próprio conjunto de instruções de máquina específicas. Essas são as únicas instruções que o processador pode realmente executar. Como existe um número finito de instruções, os projetistas do processador simplesmente listam as instruções e atribuem a elas um código binário que é usado para representá-las. Isso é semelhante à abordagem adotada ao representar dados de caracteres, conforme descrito no Capítulo 3.

A relação entre o processador e as instruções que ele pode executar é completamente integrada. A eletrônica da CPU reconhece inerentemente as representações binárias dos comandos específicos, portanto, não há uma lista real de comandos que o computador deve consultar. Em vez disso, a CPU incorpora a lista em seu design.

? Gerenciando espécies ameaçadas de extinção

Zoológicos estabeleceram populações em cativeiro de animais ameaçados de extinção para salvá-los da extinção, mas precisam ter uma boa distribuição de idades e diversidade genética para proteger as espécies contra doenças e endogamia. Um banco de dados computadorizado de todos os animais em cativeiro permite que cientistas mensurem fatores importantes que determinam o bem-estar de uma espécie. O Jardim Zoológico de Minnesota coordena o Sistema Internacional de Inventário de Espécies (ISIS), que fornece informações globais sobre mais de 2,2 milhões de animais vivos.

Cada instrução em linguagem de máquina executa apenas uma tarefa de nível muito baixo. Cada pequeno passo em um processo deve ser explicitamente codificado em linguagem de máquina. Mesmo a pequena tarefa de somar dois números deve ser dividida em etapas menores: inserir um número no acumulador, adicionar um número a ele e salvar o resultado. Em seguida, essas três instruções devem ser escritas em binário, e o programador precisa se lembrar de qual combinação de dígitos binários corresponde a qual instrução. Como mencionamos no Capítulo 1, os programadores em linguagem de máquina precisam ser muito bons com números e muito detalhistas.

No entanto, não podemos deixar você com a impressão de que apenas matemáticos podem escrever programas em linguagem de máquina. É verdade que pouquíssimos programas são escritos em linguagem de máquina hoje, principalmente porque representam um uso ineficiente do tempo do programador. Embora a maioria dos programas seja escrita em linguagens de alto nível e depois traduzida para linguagem de máquina (um processo que descreveremos mais adiante neste capítulo), todo software é na verdade implementado em código de máquina. Entender, mesmo que pouco, sobre esse nível tornará você um usuário mais informado. Além disso, essa experiência enfatiza a definição básica de um computador e faz você apreciar a facilidade com que as pessoas interagem com computadores hoje em dia.

Pep/8: Um Computador Virtual

Por definição, o código de máquina difere de máquina para máquina. Lembre-se de que, assim como cada fechadura tem uma chave específica que a abre, cada tipo de computador tem um conjunto específico de operações que pode executar, chamado de linguagem de máquina do computador. Ou seja, cada tipo de CPU tem sua própria linguagem de máquina que entende. Então, como podemos proporcionar a cada um de vocês a experiência de usar linguagem de máquina quando vocês podem estar trabalhando em máquinas diferentes? Resolvemos esse problema usando um computador virtual. Um computador virtual é uma máquina hipotética – neste caso, uma que foi projetada para conter os recursos importantes de computadores reais que queremos ilustrar. O Pep/8, projetado por Stanley Warford, é a máquina virtual que usamos aqui.[1]

* Computador virtual (máquina): Uma máquina hipotética projetada para ilustrar recursos importantes de uma máquina real.

O Pep/8 possui 39 instruções em linguagem de máquina. Isso significa que um programa para o Pep/8 deve ser uma sequência composta por uma combinação dessas instruções. Não entre em pânico: não pediremos que você entenda e memorize 39 sequências de bits binários! Planejamos apenas examinar algumas dessas instruções e não pediremos que você memorize nenhuma delas.

Recursos Importantes Refletidos no Pep/8

A unidade de memória do Pep/8 é composta por 65.536 bytes de armazenamento. Os bytes são numerados de 0 a 65.535 (decimal). Lembre-se de que cada byte contém 8 bits, portanto, podemos descrever o padrão de bits em um byte usando 2 dígitos hexadecimais. (Consulte o Capítulo 2 para mais informações sobre dígitos hexadecimais.) O comprimento da palavra no Pep/8 é de 2 bytes, ou 16 bits. Portanto, a informação que flui para dentro e para fora da unidade lógica/aritmética (ULA) tem 16 bits de comprimento.

Lembre-se do Capítulo 5 que um registrador é uma pequena área de armazenamento na ULA da CPU que contém dados especiais e valores intermediários. O Pep/8 possui sete registradores, três dos quais focaremos neste momento:

- O contador de programa (PC), que contém o endereço da próxima instrução a ser executada.
- O registrador de instruções (IR), que contém uma cópia da instrução que está sendo executada.
- O acumulador (registrador A)

O acumulador é usado para armazenar dados e os resultados das operações; é o registrador de armazenamento especial mencionado no Capítulo 5 na discussão sobre a ULA.

Sabemos que se trata de muitas informações detalhadas, mas não se desespere! Lembre-se de que nosso objetivo é dar a você uma ideia do que realmente está acontecendo no nível mais baixo do processamento do computador. Por necessidade, esse processamento rastreia muitos detalhes.

A FIGURA 6.1 mostra um diagrama da CPU e da memória do Pep8. Observe que os endereços na memória aparecem em laranja. Essa cor serve para enfatizar que os endereços em si não são armazenados na memória, mas sim que eles nomeiam os bytes individuais da memória. Nos referimos a qualquer byte específico na memória por seu endereço.

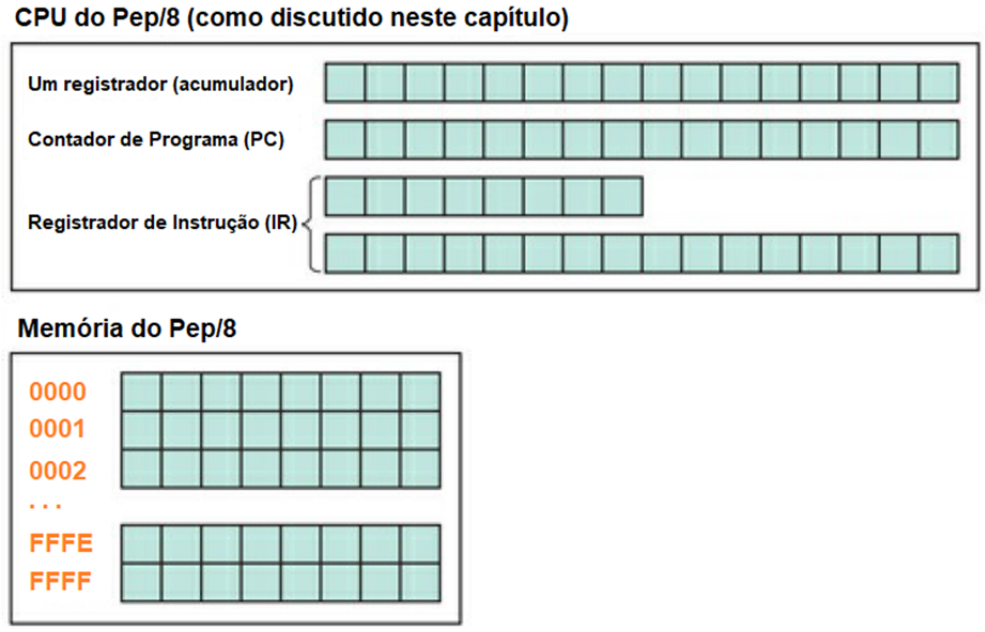


FIGURA 6.1 Arquitetura do Pep/8.

Antes de prosseguirmos, vamos revisar alguns aspectos dos números binários e hexadecimais. O maior valor decimal que pode ser representado em um byte é 255. Isso ocorre quando todos os bits são 1s: 11111111 em binário é FF em hexadecimal e 255 em decimal. O maior valor decimal que pode ser representado em uma palavra (16 bits) é 65.535. Isso ocorre quando todos os 16 bits são 1s: 1111111111111111 em binário é FFFF em hexadecimal e 65.535 em decimal. Se representarmos números positivos e negativos, perdemos um bit na magnitude (porque um é usado para o sinal), então podemos representar valores que variam de −7FFF a +7FFF em hexadecimal, ou −32.767 a +32.767 em decimal.

Essa informação é importante ao trabalhar com a máquina Pep/8. O número de bits disponíveis determina o tamanho dos números com os quais podemos trabalhar.

Formato da Instrução

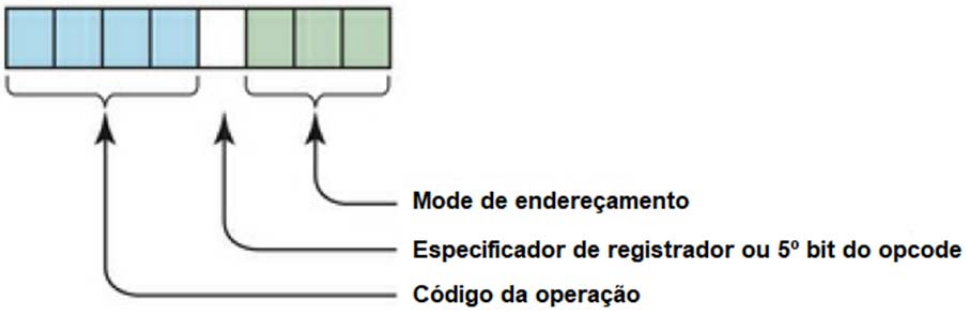
Já falamos que instruções entram no registrador de instruções (IR), são decodificadas e são executadas. Agora estamos prontos para analisar um conjunto (ou subconjunto) de instruções concretas que um computador pode executar. Primeiro, porém, precisamos examinar o formato de uma instrução em Pep/8.

A FIGURA 6.2(a) mostra o formato de uma instrução no Pep/8. Há duas partes em uma instrução: o especificador de instrução e (opcionalmente) o especificador de operando de 16 bits. O especificador de instrução indica qual operação deve ser realizada, como "adicionar um número a um valor já armazenado em um registrador", e como interpretar exatamente onde o operando está. O especificador de operando (o segundo e o terceiro bytes da instrução) contém o próprio operando ou o endereço de onde o operando deve ser encontrado. Algumas instruções não usam o especificador de operando.

O formato do especificador de instrução varia dependendo do número de bits usados para representar uma operação específica. No Pep/8, os códigos de operação (chamados opcodes) variam de 4 bits a 8 bits de comprimento. Os opcodes que abordamos têm 4 ou 5 bits de comprimento, com o quinto bit dos opcodes de 4 bits usado para especificar qual registrador usar. O especificador de registrador é 0 para o registrador A (o acumulador), que é o único registrador que usaremos. Portanto, o especificador de registrador é codificado por cores em nossos diagramas apenas quando faz parte do opcode. Veja a FIGURA 6.2(b).



(a) As duas partes de uma instrução.



(b) A parte do especificador de instrução de uma instrução.

FIGURA 6.2 Formato da instrução Pep/8.

O especificador de modo de endereçamento de 3 bits – sombreado em verde em (b), indica como interpretar a parte do operando da instrução.

- Se o modo de endereçamento for **000**, o operando está no especificador de operando da instrução. Este modo de endereçamento é chamado de imediato (i).
- Se o modo de endereçamento for **001**, o operando é o endereço de memória nomeado no especificador de operando. Este modo de endereçamento é chamado de direto (d).

Outros modos de endereçamento também existem, mas não os abordaremos aqui. A distinção entre o modo de endereçamento imediato e o modo de endereçamento direto é muito importante porque determina onde os dados envolvidos na operação são armazenados ou devem ser armazenados. Veja a FIGURA 6.3. Os locais que contêm endereços estão sombreados em laranja; operandos estão sombreados em cinza.

Instruções que não possuem um operando (dados a serem manipulados) são chamadas de instruções unárias; elas não possuem um especificador de operando. Ou seja, instruções unárias têm apenas 1 byte de comprimento, em vez de 3 bytes.

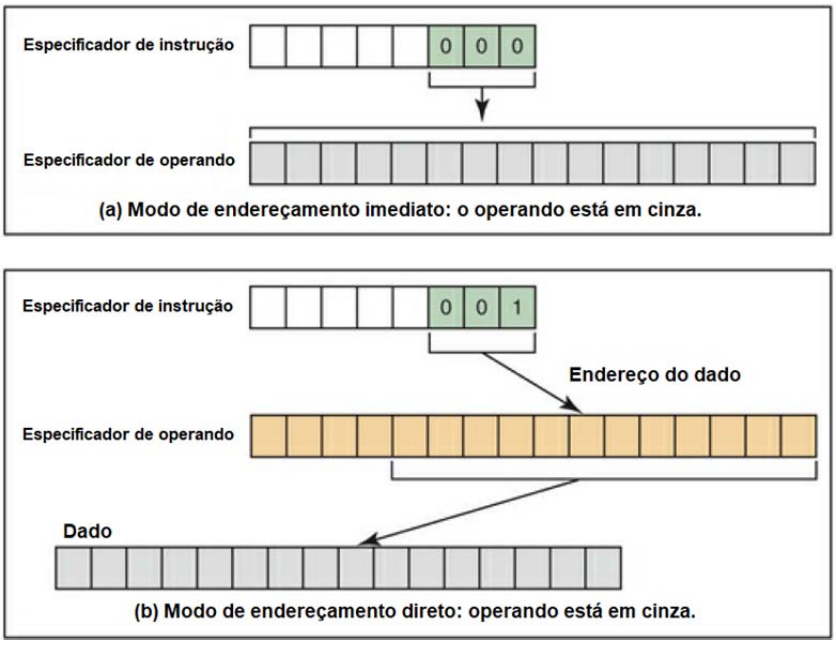


FIGURA 6.3 A diferença entre o modo de endereçamento imediato e o modo de endereçamento direto.

Alguns Exemplos de Instruções

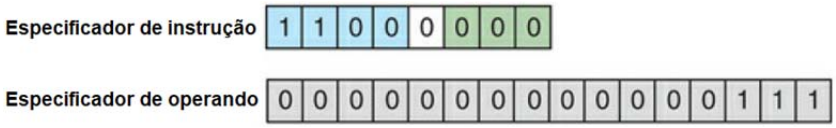
Vamos analisar algumas instruções específicas isoladamente e, em seguida, juntá-las para escrever um programa. A FIGURA 6.4 contém o código de operação de 4 bits (ou opcode) para as operações que estamos abordando.

- **0000** - Parar a execução: Durante o ciclo de busca-execução, quando o código da operação é todo zero, o programa é interrompido. Parar é uma instrução unária, portanto, ocupa apenas um byte. Os três bits mais à direita no byte são ignorados.
- **1100** - Carregar o operando no registrador A: Esta instrução carrega uma palavra (dois bytes) no registrador A. O especificador de modo determina onde a palavra está localizada. Portanto, o código de operação de carga tem significados diferentes dependendo do especificador de modo de endereçamento. O especificador de modo determina se o valor a ser carregado está na parte do operando da instrução (o segundo e o terceiro bytes da instrução) ou no local nomeado no operando.

Seguem exemplos concretos de cada uma dessas combinações. Segue a primeira instrução de 3 bytes:

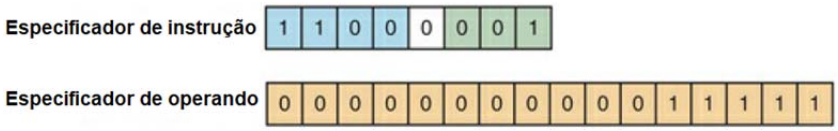
Opcode	Instrução
0000	Parar a execução.
1100	Carregar o operando dentro do registrador A.
1110	Carregar o conteúdo do registrador A dentro do operando.
0111	Adicionar o operando ao registrador A.
1000	Subtrair o operando do registrador A.
01001	Entrar caractere no operando.
01010	Saída de caractere do operando.

FIGURA 6.4 Subconjunto de instruções Pep/8.



O modo de endereçamento é imediato, o que significa que o valor a ser carregado no registrador A está no especificador de operando. Ou seja, os dados estão no especificador de operando, portanto, estão sombreados em cinza. Após a execução desta instrução, o conteúdo do segundo e do terceiro bytes da instrução (o especificador de operando) seria carregado no registrador A (o acumulador). Ou seja, o registrador A conteria 0007 e o conteúdo original (anterior) de A seria perdido.

Aqui está outra instrução de carregamento:



O modo de endereçamento é direto, o que significa que o operando em si não está no especificador de operando (segundo e terceiro bytes da instrução); em vez disso, o especificador de operando contém o endereço (laranja) de onde o operando reside na memória. Assim, quando esta instrução é executada, o conteúdo da posição 001F seria carregado no registrador A. Observe que sombreamos os bits que representam um endereço de memória em laranja, assim como usamos laranja para outros endereços. O registrador A contém uma palavra (2 bytes), portanto, quando um endereço é usado para especificar uma palavra (em vez de um único byte), como neste caso, o endereço fornecido é o do byte mais à esquerda na palavra. Assim, o conteúdo das posições adjacentes 001F e 0020 é carregado no registrador A. O conteúdo do operando (001F e 0020) não é alterado.

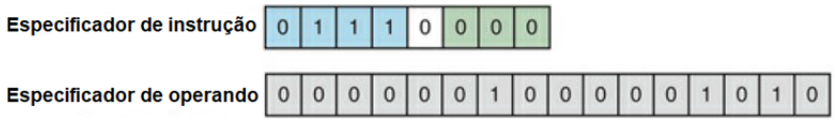
- **1110** - Armazena o registrador A no operando: Esta instrução armazena o conteúdo do registrador A no local especificado no operando.

Esta instrução armazena o conteúdo do registrador A na palavra que começa no local 000A. É inválido usar um modo de endereçamento imediato com um opcode de armazenamento; ou seja, não podemos tentar armazenar o conteúdo de um registrador no especificador de operando.

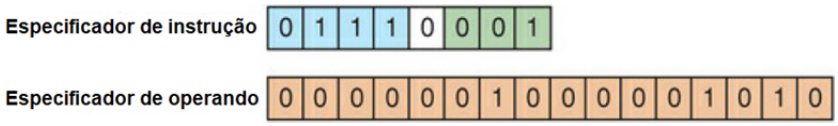
? Traga um martelo

Quando perguntaram a Jan Hein Donner, o grande mestre de xadrez holandês, como ele se prepararia para uma partida de xadrez contra um computador como o Deep Blue da IBM, ele respondeu: "Eu levaria um martelo".[2]

- **0111** - Adiciona o operando ao registrador A: Assim como a operação de carregamento, a operação de adição usa o especificador do modo de endereçamento, oferecendo interpretações alternativas. As duas alternativas para esta instrução são mostradas abaixo, com a explicação após cada instrução.



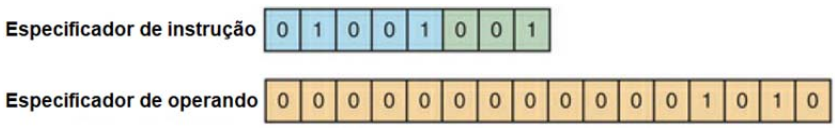
O conteúdo do segundo e terceiro bytes da instrução (o especificador do operando) é adicionado ao conteúdo do registrador A (20A em hexadecimal). Assim, sombreamos o especificador do operando para mostrar que se trata de dados.



Como o especificador do modo de endereço é direto, o conteúdo do operando especificado no segundo e terceiro bytes da instrução (local 020A) é adicionado ao registrador A.

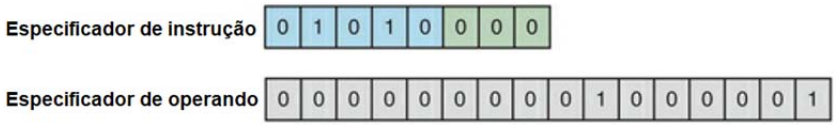
- **1000** - Subtrai o operando: Esta instrução é semelhante à operação de adição, exceto que o operando é subtraído do registrador A em vez de adicionado. Assim como nas operações de carregamento e adição, há variações desta instrução dependendo do modo de endereçamento.

- **01001** - Entrada de caracteres no operando: Esta instrução permite que o programa insira um caractere ASCII do dispositivo de entrada enquanto o programa está em execução. Somente o endereçamento direto é permitido, portanto, o caractere é armazenado no endereço mostrado no especificador do operando.

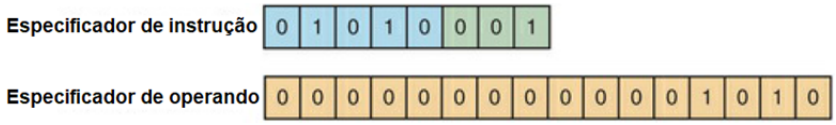


Esta instrução lê um caractere ASCII do dispositivo de entrada e o armazena na posição 000A.

- **01010** - Saída de caracteres do operando: Esta instrução envia um caractere ASCII para o dispositivo de saída enquanto o programa está em execução. O modo de endereçamento pode ser imediato ou direto.



Como o endereçamento imediato é especificado, esta instrução escreve o caractere ASCII armazenado no especificador de operando. O especificador de operando contém 1000001, que é 41 em hexadecimal e 65 em decimal. O caractere ASCII correspondente a esse valor é 'A', portanto, a letra 'A' é escrita na tela.



Como o endereçamento direto é usado, esta instrução escreve o caractere ASCII armazenado no local nomeado no especificador de operando, local 000A. O que é escrito? Não podemos dizer a menos que conheçamos o conteúdo do local 000A. O caractere ASCII correspondente a tudo o que estiver armazenado nesse local é impresso.

6.3 Um Exemplo de Programa

Agora estamos prontos para escrever nosso primeiro programa em linguagem de máquina: vamos escrever "Hello" na tela. Há seis instruções neste programa: cinco para escrever um caractere e uma para indicar o fim do processo. A instrução para escrever um caractere na tela é 0101, a operação "Saída de caractere do operando". Devemos armazenar os caracteres na memória e escrevê-los usando o modo de endereçamento direto, ou devemos apenas armazená-los no especificador de operando e usar o modo de endereçamento imediato? Vamos usar o endereçamento imediato aqui e deixar o endereçamento direto como exercício. Isso significa que o especificador do modo de endereçamento é 000 e o código ASCII da letra vai para o terceiro byte da instrução.

Ação	Instrução Binária	Instrução em Hexadecimal
Write 'H'	**01010000** 0000000001001000	50 0048
Write 'e'	**01010000** 0000000001100101	50 0065
Write 'l'	**01010000** 0000000001101100	50 006C
Write 'l'	**01010000** 0000000001101100	50 006C
Write 'o'	**01010000** 0000000001101111	50 006F
Stop	**0000**0000	00

O programa em linguagem de máquina é mostrado em binário na segunda coluna e em hexadecimal na terceira coluna. Devemos construir o especificador de operação em binário porque ele é composto por um opcode de 4 bits, um especificador de registrador de 1 bit e um especificador de modo de endereçamento de 3 bits. Assim que tivermos os 8 bits completos, podemos converter a instrução para hexadecimal. Alternativamente, podemos construir o especificador de operando diretamente em hexadecimal.

Usamos aspas duplas ao nos referirmos a uma coleção de caracteres como "Hello" e aspas simples ao nos referirmos a um único caractere. Esse padrão é comumente usado em algumas linguagens de programação, portanto, seguimos essa convenção aqui.

Simulação Manual

Vamos simular a execução deste programa seguindo as etapas do ciclo de busca-execução. Esses rastreamentos manuais realmente direcionam as etapas que o computador executa.

Relembre as quatro etapas do ciclo de busca-execução:

1. Buscar a próxima instrução (do local indicado no contador de programa).
2. Decodificar a instrução (e atualizar o contador de programa).
3. Obter dados (operando), se necessário.
4. Executar a instrução.

Há seis instruções em nosso programa exemplo. Vamos supor que elas estejam em posições contíguas na memória, com a primeira instrução armazenada nas posições de memória 0000 – 0002. A execução começa carregando 0000 no contador de programa (PC). Em cada etapa da execução, vamos examinar o CP (mostrado em laranja) e o registrador de instruções (IR). O programa não acessa o registrador A, portanto, não nos preocupamos em mostrá-lo. Ao final da primeira busca, o CP e o IR se parecem com o diagrama a seguir. (Continuamos a usar cores para enfatizar os endereços, opcode, especificador de modo de endereço e dados.) Observe que o contador de programa é incrementado assim que a instrução é acessada.



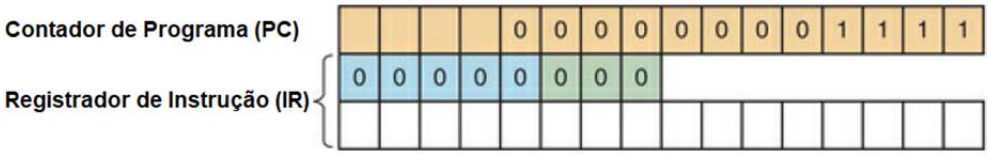
Esta instrução é decodificada como uma instrução "Escrever caractere na saída" usando o modo de endereçamento imediato. Como esta instrução ocupa 3 bytes, o PC é incrementado em 3. Os dados são recuperados do especificador de operando no IR, a instrução é executada e 'H' é escrito na tela.

A segunda busca é executada e o PC e o IR são os seguintes:



Esta instrução é decodificada como outra instrução "Escrever caractere na saída" usando o modo de endereçamento imediato. A instrução ocupa 3 bytes, então o PC é novamente incrementado em 3. Os dados são recuperados, a instrução é executada e 'e' é escrito na tela.

As próximas três instruções são executadas exatamente da mesma maneira. Após o 'o' ter sido escrito, o PC e o IR têm a seguinte aparência:



O opcode é decodificado como uma instrução "Stop", de modo que o conteúdo do modo de endereçamento e o especificador de operando são ignorados. Nesse ponto, o ciclo de busca-execução é interrompido.

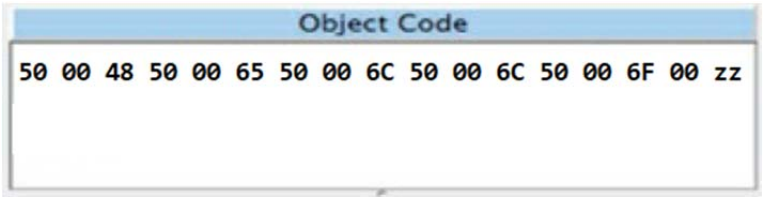
? Golpe de cheque nigeriano

Em junho de 2008, Edna Fiedler, de Olympia, Washington, foi condenada a 2 anos de prisão e 5 anos de liberdade condicional supervisionada em um golpe envolvendo um cheque nigeriano de US$ 1 milhão. Nesse golpe, uma mensagem em inglês rudimentar implorava por ajuda financeira do destinatário. Em todos os casos, o remetente era um alto funcionário que tinha milhões escondidos em um local inacessível. Se o destinatário transferisse dinheiro para a fuga do funcionário de seu país devastado, ele ou ela recebia uma parte do dinheiro. O prejuízo médio para as vítimas desse golpe era de mais de US$ 5.000.

Simulador Pep/8

Lembre-se de que as instruções são escritas na linguagem de máquina do Pep/8, que não corresponde à linguagem de máquina de nenhuma CPU específica. Acabamos de simular manualmente o programa. Podemos executá-lo no computador? Sim, podemos. O Pep/8 é uma máquina virtual (hipotética), mas temos um simulador para a máquina. Ou seja, temos um programa que se comporta exatamente como a máquina

virtual Pep/8. Para executar um programa, inserimos o código hexadecimal byte por byte, com exatamente um espaço em branco entre cada byte, e encerramos o programa com "zz". O simulador reconhece dois 'z's como o fim do programa. Aqui está uma captura de tela do programa em linguagem de máquina do Pep/8:

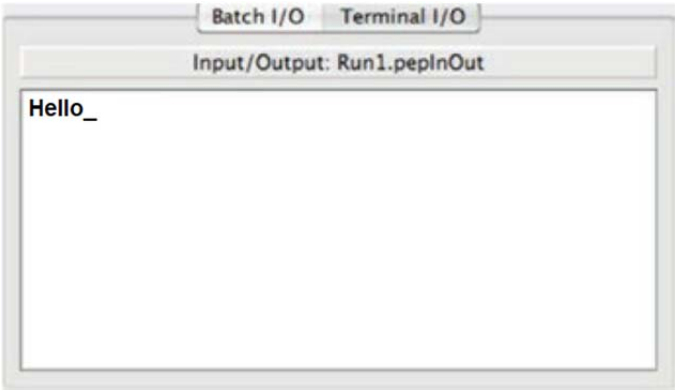


Vamos percorrer as etapas necessárias para inserir e executar um programa. Suponhamos que o simulador Pep/8 esteja instalado. Para iniciar o programa, clique no ícone do Pep/8. Uma das várias telas pode aparecer, mas cada uma contém uma seção marcada como "Object Code" (ou "Código Objeto"). Insira seu programa nesta janela, conforme descrito anteriormente. Agora você está pronto para executar seu programa. Vá para a barra de menus. Apresentamos ao lado uma imagem da parte necessária.



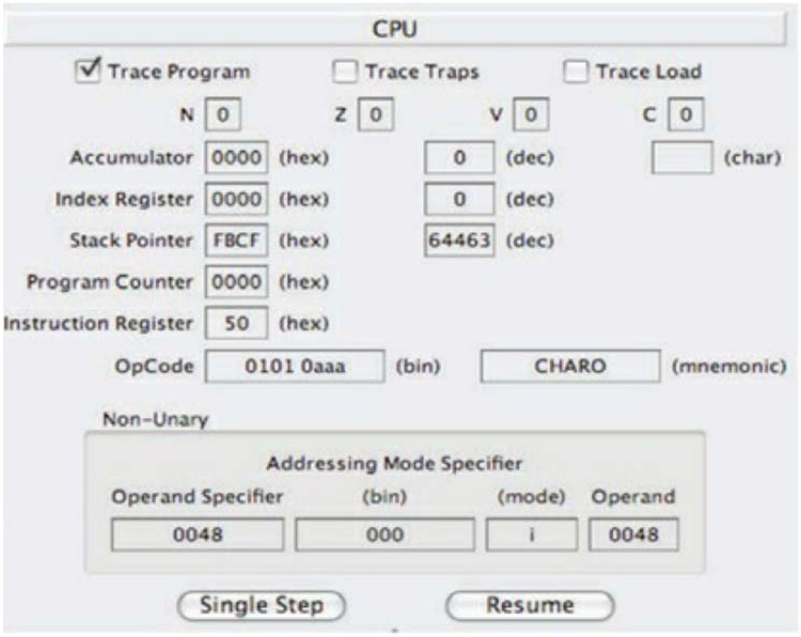
Clique no meio desses três ícones, que chama o loader (carregador). Após clicar neste ícone, seu programa é carregado na memória do Pep/8.

Certifique-se de que o botão Terminal I/O esteja escurecido (pressionado). Agora clique no ícone mais à direita, que é o botão executar. O programa é executado e "Hello" aparece na janela de saída. Para tudo o que fizermos neste capítulo, o botão Terminal I/O deve estar escurecido. Esta área é onde você insere e exibe valores.



* Loader: Um software que pega um programa em linguagem de máquina e o coloca na memória.

O Pep/8 possui um recurso que permite observar o que está acontecendo na CPU à medida que cada instrução é executada. Aqui está uma captura de tela da CPU após o programa ter sido carregado. Observe que a caixa de seleção "Trace Programa" (ou Rastrear Programa) foi marcada. Esta tela inclui várias caixas que não abordamos, mas você pode ver facilmente os rótulos "Program Counter" (ou Contador de Programa), "Instruction Register" (ou Registrador de Instrução) e "OpCode" (Código da Operação).



Quando a opção "Trace Program" estiver marcada, pressione o botão "Single Step" (Passo a Passo) e a primeira instrução será executada. Continue pressionando o botão de passo-a-passo e você poderá ver os valores do registrador mudarem.

Antes de encerrarmos nosso exemplo de código de máquina, vamos inserir duas letras e imprimi-las em ordem inversa. Podemos escolher um local para inserir a entrada, pois ela é lida em algum lugar além do código. Neste caso, escolhemos "0F" e "12". Usamos o modo de endereçamento direto.

Ação	Instrução Binária	Instrução em Hexadecimal
Entre uma letra na localização F	**01001001** 0000000000001111	49 000F
Entre uma letra em F + 1	**01001001** 0000000000001000	49 0010
Escreva a segunda letra	**01010001** 0000000000001000	51 0010
Escreva a primeira letra	**01010001** 0000000000001111	51 000F
Stop	**00000000**	00

Aqui está o código objeto e a janela de saída após inserir 'A' e 'B':



6.4 Linguagem Assembly

Como apontamos no Capítulo 1, as primeiras ferramentas desenvolvidas para auxiliar o programador foram as linguagens de montagem. As linguagens de montagem atribuem códigos de letras mnemônicas a cada instrução em linguagem de máquina. O programador usa esses códigos de letras no lugar de dígitos binários. As instruções em uma linguagem de montagem são muito parecidas com aquelas que usaríamos para dizer a alguém como fazer um cálculo em uma calculadora portátil.

* Linguagem de montagem: Uma linguagem de programação de baixo nível na qual um mnemônico representa cada uma das instruções em linguagem de máquina para um computador específico.

Como todo programa executado em um computador deve eventualmente estar na forma da linguagem de máquina do computador, um programa chamado assembler (ou montador) lê cada uma das instruções na forma mnemônica e as traduz para o equivalente em linguagem de máquina. Além disso, como cada tipo de computador possui uma linguagem de máquina diferente, existem tantas linguagens de montagem e tradutores quantos tipos de máquinas.

* Assembler (ou Montador): Um programa que traduz um programa em linguagem de montagem em código de máquina.

Linguagem Assembly do Pep/8

O objetivo desta seção não é torná-lo um programador em linguagem assembly, mas sim, fazê-lo apreciar as vantagens da programação em linguagem assembly em relação à codificação de máquina. Com esse objetivo em mente, abordamos apenas alguns dos recursos da linguagem assembly do Pep/8. Começamos examinando as mesmas operações que vimos nas seções anteriores, além de três outras operações úteis. Na linguagem assembly do Pep/8, há um opcode diferente para cada registrador, o operando é especificado por "0x" e o valor hexadecimal, e o especificador do modo de endereçamento é indicado pelas letras 'i' ou 'd'.

Você se perguntou por que não fizemos nenhuma aritmética em linguagem de máquina? Bem, a saída foi definida apenas para caracteres. Se tivéssemos feito aritmética, teríamos que converter os números para o formato de caracteres para ver os resultados, e isso é mais complexo do que desejaríamos obter. A linguagem assembly do Pep/8 fornece os mnemônicos DECI e DECO, que nos permitem fazer entrada e saída decimais. Essa terminologia é um tanto enganosa, no entanto, porque essas operações, na verdade, envolvem chamadas para uma série de instruções nos bastidores.

Mnemônico	Operando, Especificador de Modo	Significado da Instrução
STOP		Interrompa a execução.
LDA LDA	0x008B, i 0x008B, d	Carregue 008B no registrador A. Carregue o conteúdo da posição 8B no registrador A.
STA	0x008B, d	Armazene o conteúdo do registrador A na posição 8B.
ADDA ADDA	0x008B, i 0x008B, d	Adicione 008B ao registrador A. Adicione o conteúdo da posição 8B ao registrador A.
SUBA SUBA	0x008B, i 0x008B, d	Subtraia 008B do registrador A. Subtraia o conteúdo da posição 8B do registrador A.
BR		Desvie para a posição especificada no especificador de operando.
CHARI	0x008B, d	Lê um caractere e armazena-o na posição 8B.
CHARO	0x008B, i 0x000B, d	Escreva o caractere 8B. Escreva o caractere armazenado na posição 0B.
DECI DEC0 DEC0	0x008B, d 0x008B, i 0x008B, d	Lê um número decimal e armazena-o na posição 8B. Escreva o número decimal 139 (8B em hexadecimal). Escreva o número decimal armazenado na posição 8B.

Diretivas do Assembler

Em um programa escrito em linguagem de máquina, cada instrução é armazenada na memória e então executada. Começando com as linguagens de montagem, a maioria das linguagens de programação possui dois tipos de instruções: instruções a serem traduzidas e instruções para o programa tradutor. Aqui estão algumas diretivas de montagem úteis para o montador Pep/8 – ou seja, instruções para o montador. Essas instruções para o montador também são chamadas de pseudo-operações.

Pseudo OP	Argumento	Significado
.ASCII	"Str\x00"	Representa uma sequência de bytes ASCII.
.BLOCK	Número de bytes	Cria um bloco de bytes.
.WORD	Valor	Cria uma palavra e armazena um valor nela.
.END		Sinaliza o fim do programa em linguagem assembly.

* Diretivas de montagem: Instruções para o programa tradutor.

Versão em Linguagem Assembly do Programa Hello

Vamos dar uma olhada no programa em linguagem assembly que escreve "Hello" na tela. A linguagem assembly do Pep/8 nos permite especificar diretamente o caractere a ser exibido e adicionar um comentário ao lado da instrução. Um comentário é um texto escrito para o leitor humano do programa que explica o que está acontecendo. Comentários são uma parte essencial da escrita de qualquer programa. O montador ignora tudo, desde o ponto e vírgula até o final da linha; é um comentário. Estão coloridos para destacar.

* Comentário: Texto explicativo para o leitor humano.

```
CHARO   0x0048, i;   Output an 'H'
CHARO   0x0065, i;   Output an 'e'
CHARO   0x006C, i;   Output an 'l'
CHARO   0x006C, i;   Output an 'l'
CHARO   0x006F, i;   Output an 'o'
STOP
.END
```

Este código é inserido na janela "Source Code". O ícone à esquerda do ícone de carregamento é o ícone do montador. Clique neste ícone e o código-objeto para o qual o programa é traduzido aparece na janela "Object Code". A janela "Assembler Listing" mostra o endereço ao qual uma instrução foi atribuída, o código-objeto e o código em linguagem assembly; É mostrado aqui (ao lado):

Assembler Listing

Addr	Code	Mnemon	Operand	Comment
0000	500048	CHARO	0x0048, i	
0003	500065	CHARO	0x0065, i	
0006	50006C	CHARO	0x006C, i	
0009	50006C	CHARO	0x006C, i	
000C	50006F	CHARO	0x006F, i	
000F	00	STOP		

O processo de execução de um programa codificado em linguagem assembly é ilustrado na FIGURA 6.5. A entrada para o montador é um programa escrito em linguagem assembly. A saída do montador é um programa escrito em código de máquina. Você pode ver por que a criação da linguagem assembly

representou um passo tão importante na história das linguagens de programação: Ele removeu muitos detalhes da programação em linguagem de máquina, abstraindo as instruções em palavras. Embora tenha adicionado uma etapa ao processo de execução de um programa (a tradução do assembly para código de máquina), essa etapa extra vale o esforço para facilitar a vida do programador.

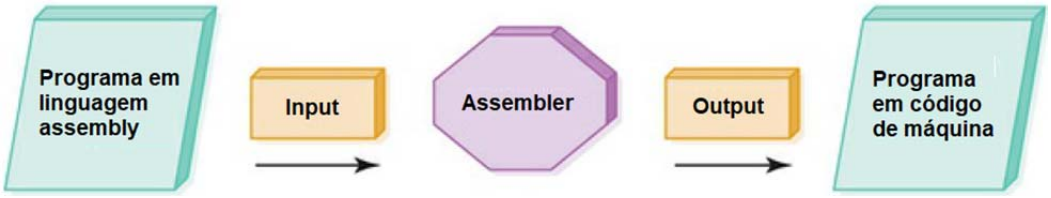


FIGURA 6.5 Processo de montagem.

Um Novo Programa

Vamos aumentar a complexidade e escrever um programa para ler três números e escrever sua soma. Como faríamos essa tarefa manualmente? Se tivéssemos uma calculadora, primeiro limparíamos o total; isto é, zeraríamos a soma, ou seja, inicialmente teríamos total igual a zero. Em seguida, obteríamos o primeiro número e o somaríamos ao total, obteríamos o segundo número e o somaríamos ao total e, finalmente, obteríamos o terceiro número e o somaríamos ao total. O resultado seria o que está no total, ou seja, no acumulador da calculadora. Podemos modelar nosso programa de computador com base nessa solução manual.

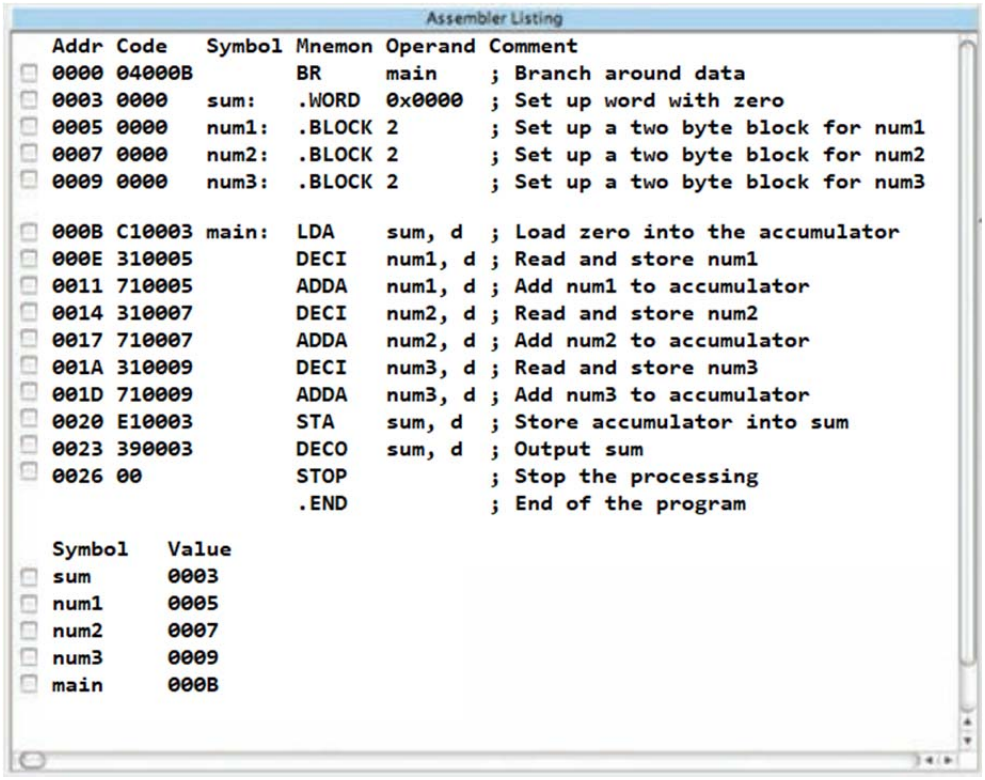
O problema mais complexo é que precisamos associar quatro identificadores a posições na memória, e isso requer saber quantas posições o próprio programa ocupa – isto é, se colocarmos os dados no final do programa. Vamos facilitar esse processo colocando nossos dados antes do programa. Podemos começar a associar identificadores a posições de memória começando pela posição 0001 e fazer com que o ciclo de busca-execução pule essas posições para continuar com o programa. De fato, podemos atribuir identificadores às posições de memória e usar esses nomes posteriormente no programa. Configuramos o espaço para a soma usando a pseudo-operação .WORD para que possamos definir o conteúdo como 0. Configuramos o espaço para os três números usando a pseudo-operação .BLOCK.

```
        BR          main        ; Branch around data
sum:    .WORD       0x0000      ; Set up word with zero
num1:   .BLOCK 2                ; Set up a two byte block for num1
num2:   .BLOCK 2                ; Set up a two byte block for num2
num3:   .BLOCK 2                ; Set up a two byte block for num3

main:   LDA         sum, d      ; Load zero into the accumulator
        DECI        num1, d     ; Read and store num1
        ADDA        num1, d     ; Add num1 to accumulator
        DECI        num2, d     ; Read and store num2
        ADDA        num2, d     ; Add num2 to accumulator
        DECI        num3, d     ; Read and store num3
        ADDA        num3, d     ; Add num3 to accumulator
        STA         sum, d      ; Store accumulator into sum
        DECO        sum, d      ; Output sum
        STOP                    ; Stop the processing
        .END                    ; End of the program
```

A seguir apresentamos a listagem do assembler para este programa, seguida por uma captura de tela da janela de Entrada/Saída após a execução do programa. Observe que o usuário digita os três valores (23, 14, 2) e o programa imprime a soma deles (39).





```
Assembler Listing
Addr Code   Symbol Mnemon Operand Comment
0000 04000B        BR     main    ; Branch around data
0003 0000   sum:   .WORD  0x0000  ; Set up word with zero
0005 0000   num1:  .BLOCK 2       ; Set up a two byte block for num1
0007 0000   num2:  .BLOCK 2       ; Set up a two byte block for num2
0009 0000   num3:  .BLOCK 2       ; Set up a two byte block for num3

000B C10003 main:  LDA    sum, d  ; Load zero into the accumulator
000E 310005        DECI   num1, d ; Read and store num1
0011 710005        ADDA   num1, d ; Add num1 to accumulator
0014 310007        DECI   num2, d ; Read and store num2
0017 710007        ADDA   num2, d ; Add num2 to accumulator
001A 310009        DECI   num3, d ; Read and store num3
001D 710009        ADDA   num3, d ; Add num3 to accumulator
0020 E10003        STA    sum, d  ; Store accumulator into sum
0023 390003        DECO   sum, d  ; Output sum
0026 00            STOP           ; Stop the processing
                   .END           ; End of the program

Symbol   Value
sum      0003
num1     0005
num2     0007
num3     0009
main     000B
```

Um Programa com Ramificação

Mostramos que o contador de programa pode ser alterado com uma instrução BR que define o contador de programa para o endereço de uma instrução a ser executada em seguida. Existem outras maneiras de alterar o fluxo de controle do programa? Podemos fazer uma pergunta e realizar uma ou outra ação com base na resposta à nossa pergunta? Claro – vamos ver como. Aqui estão dois opcodes úteis e seus significados:

Mnemônico	Operando, Especificador de Modo	Significado da Instrução
BRLT	i	Defina o PC para que o operando do registrador A seja menor que zero.
BREQ	i	Defina o PC para que o operando do registrador A seja igual a zero.

? Quem é Genevieve Bell?

A Dra. Bell é uma antropóloga na Intel, a maior produtora mundial de semicondutores. Ela é diretora de pesquisa de experiência do usuário no Intel Labs, o braço de pesquisa da empresa. Ela lidera um grupo de cerca de 100 cientistas sociais e designers que viajam pelo mundo, observando como as pessoas usam a tecnologia em suas casas e em público.[3]

Por exemplo:

```
LDA num1, d      ; Load num1 into A register
BRLT lessThan    ; Branch to lessThan if num1 is less than 0
```

Se o valor armazenado em num1 for negativo quando carregado no registrador A, o PC é definido para o local lessThan. Se o valor não for negativo, o PC permanece inalterado. Vamos alterar o programa anterior para que ele imprima a soma se for positiva e exiba uma mensagem de erro se for negativa. Onde o teste deve ser feito? Pouco antes do conteúdo da resposta ser armazenado no local "sum", podemos testar o registrador A e imprimir 'E' se for negativo.

Podemos usar a instrução BRLT para testar se a soma é negativa. Se o registrador A for negativo, o operando ao lado da instrução BRLT substitui o conteúdo do contador do programa para que a próxima instrução venha de lá. Precisamos dar um nome à instrução para que possamos desviar para ela. Vamos chamar a instrução que imprime a mensagem de erro de negMsg. Quando a mensagem de erro for escrita, devemos retornar a linha que diz STOP, o que significa que devemos nomear essa linha. Vamos chamá-la de "finish".

Aqui está o código-fonte deste programa alterado. Observe que reduzimos o número de comentários. Se o comentário apenas duplicar a instrução, pode ser uma distração em vez de uma ajuda.

```
          BR       main         ; Branch around data
sum:      .WORD    0x0000       ; Set up word with zero
num1:     .BLOCK   2            ; Set up a two byte block for num1
num2:     .BLOCK   2            ; Set up a two byte block for num2
num3:     .BLOCK   2            ; Set up a two byte block for num3

negMsg:   CHARO    0x0045, i    ; Print ‘E’
          BR       finish       ; Branch to STOP instruction
main:     LDA      sum, d       ; Load zero into the accumulator
          DECI     num1, d      ; Read and add three numbers
          ADDA     num1, d
          DECI     num2, d
          ADDA     num2, d
          DECI     num3, d
          ADDA     num3, d
          BRLT     negMsg       ; Branch to negMsg if A < 0
          STA      sum, d       ; Store accumulator into sum
          DECO     sum, d       ; Output sum
finish:   STOP                  ;
          .END                  ;
```

A seguir, a listagem do montador, seguida de uma captura de tela da entrada e saída:

Assembler Listing

```
Addr Code   Symbol  Mnemon Operand    Comment
0000 040011         BR     main       ;  Branch around adat
0003 0000   sum:    .WORD  0x0000     ;  Set up word with zero
0005 0000   num1:   .BLOCK 2          ;  Set up a two byte block for num1
0007 0000   num2:   .BLOCK 2          ;  Set up a two byte block for num2
0009 0000   num3:   .BLOCK 2          ;  Set up a two byte block for num3
000D 500045 negMsg: CHARO  0x0045, i  ;  Print 'E'
000E 04002F         BR     finish     ;  Branch to STOP instruction
0011 C10003 main:   LDA    sum, d     ;  Load zero into the accumulator

0014 310005         DECI   num1, d    ;  Read and add three mumbers
0017 710005         ADDA   num1, d    ;
001A 310007         DECI   num2, d    ;
001D 710007         ADDA   num2, d    ;
0020 310009         DECI   num3, d    ;
0023 710009         ADDA   num3, d    ;
0026 08000B         BRLT   segMsg     ;  Branch to negMsg if A < 0
0029 E10003         STA    sum, d     ;  Store result into sum
002C 390003         DECO   sum, d     ;  Output sum
002F 00     funish: STOP               ;
                    .END               ;


Symbol Value
sum    0003
num1   0005
num2   0007
num3   0009
negMsg 000B
main   0011
finish 002F
```

Input/Output

```
2
-3
-1
E
```

Um Programa com um Laço

E se quisermos ler e somar quatro valores? Cinco valores? Qualquer número de valores? Podemos inserir quantos valores queremos somar (**limit**) e escrever o código para ler e somar esses valores. Fazemos isso criando um laço de contagem – uma seção de código que se repete um número especificado de vezes.

Dentro do código do laço, um valor é lido e somado. Como podemos controlar quantos valores lemos? Podemos criar uma marca de hash cada vez que repetimos o loop e comparar a soma das marcas de hash com o número de vezes que desejamos repetir o loop. Na verdade, nossa marca de hash é um lugar na memória onde armazenamos um 0; vamos chamá-lo de **counter**. Cada vez que o loop é repetido, adicionamos um 1 a esse lugar na memória. Quando **counter** é igual a **limit**, encerramos a leitura e a contagem.

Na próxima seção, descreveremos o pseudocódigo, uma maneira menos prolixa de explicar o que fazemos em situações de ramificação e loop. Por enquanto, aqui está o código que lê o número de valores de dados a serem lidos e somados, lê e soma os valores e imprime o resultado:

```
            BR      main            ; Branch around data
sum:        .WORD   0x0000          ; Set up word with zero
num:        .BLOCK  2               ; Set up a block for num
limit:      .BLOCK  2               ; Set up a block for limit
counter:    .WORD   0x0000          ; Set up counter

main:       DECI    limit, d        ; Input limit
loop:       DECI    num, d          ; Read and sum limit mumbers
            LDA     num, d
            ADDA    sum, d
            STA     sum, d
quit:       LDA     counter, d      ; Load counter into A register
            ADDA    1, i            ; Add one to counter
            STA     counter, d
            CPA     limit, d        ; Compare conter and limit
            BREQ    quit            ; Go to quit if equal
            BR      loop            ; Repeat loop
            DECO    sum, d          ; Output sum
            STOP
            .END
```

Segue a listagem do montador, e depois uma captura de tela de uma execução:

```
Assembler Listing
Addr Code   Symbol   Mnemon Operand  Comment
0000 04000B          BR     main       ; Branch around data
0003 0000   sum:     .WORD  0x0000     ; Set word with zero
0005 0000   num:     .BLOCK 2          ; Set up a block for num
0007 0000   limit:   .BLOCK 2          ; Set up a block for limit
0009 0000   counter: .WORD  0x0000     ; Set up counter

000B 310007 main:    DECI   limit, d   ; Input limit
000E 310005 loop:    DECI   num, d     ; Read and sum limit mumbers
0011 C10005          LDA    num, d     ;
0014 710003          ADDA   sum, d     ;
0017 E10003          STA    sum, d     ;
001A C10009          LDA    counter, d ; Load counter into A register
001D 700001          ADDA   1, i       ; Add one to counter
0020 E10009          STA    counter, d ;
0023 B10007          CPA    limit, d   ; Compare counter and limit
0026 0A002C          BREQ   quit       ; Go to quit if equal
0029 04000E          BR     loop       ; Repeat loop
002C 390003 quit:    DECO   sum, d     ; Output sum
002F 00              STOP
                     .END

Symbol  Value
sum     0003
num     0005
limit   0007
counter 0009
main    000B
loop    000E
quit    002C

No errors. Successful assembly
```

```
Input/Output
3
2
4
6
12
```

6.5 Expressando Algoritmos

Nas seções anteriores, escrevemos programas para escrever uma saudação, ler números e escrevê-los em ordem reversa, somar três números e imprimir uma mensagem de erro se a soma for negativa, inserir um valor e ler e somar essa quantidade de números. Expressamos a solução para cada problema em forma de parágrafo e, em seguida, escrevemos o código no editor do assembly. Em computação, o plano para uma solução é chamado de **algoritmo**. Como você viu, passar do problema em forma de parágrafo para o código nem sempre é um processo claro. Pseudocódigo é uma linguagem que nos permite expressar algoritmos de forma mais clara.

* Algoritmo: Um plano ou esboço de uma solução; uma sequência lógica de etapas que resolve um problema.

* Pseudocódigo: Uma linguagem projetada para expressar algoritmos.

Funcionalidade do Pseudocódigo

No Capítulo 1, falamos sobre as camadas de linguagem que envolvem a máquina propriamente dita. Não mencionamos o pseudocódigo naquele momento porque não é uma linguagem de computador, mas sim uma linguagem semelhante a uma taquigrafia que as pessoas usam para expressar ações. Não há regras gramaticais especiais para pseudocódigo, mas para expressar ações, devemos ser capazes de representar os seguintes conceitos.

? O que foi o Projeto Genoma Musical?

Em 2002, Will Glaser, Jon Craft e Tim Westergren fundaram a empresa Savage Beast Technologies e criaram o Music Genome Project (ou Projeto Genoma Musical). O projeto, criado para capturar "a essência da música no nível mais fundamental", utiliza centenas de atributos musicais ou "genes" para descrever partituras, bem como uma complexa fórmula matemática para analisá-las. O projeto analisou dezenas de milhares de partituras e artistas diversos em busca de atributos como melodia, harmonia e ritmo, instrumentação, orquestração, arranjo e letras.

Variáveis

Nomes que aparecem em algoritmos de pseudocódigo que referem-se a locais na memória onde os valores são armazenados. O nome deve refletir a função do conteúdo no algoritmo.

Atribuição

Se tivermos variáveis, devemos ter uma maneira de inserir um valor em uma delas.

Usamos a instrução `Set sum to 0` para armazenar um valor na variável **sum**.

Outra maneira de expressar o mesmo conceito usa uma seta inversa (←), com `sum ← 1`.

Se atribuirmos valores a variáveis com a instrução de atribuição, como os acessamos posteriormente? Acessamos valores em **sum** e **num** na seguinte instrução:

`Set sum to sum + num` ou `Sum ← sum + num`

O valor armazenado em **sum** é adicionado ao valor em **num** e o resultado é armazenado novamente em **sum**. Assim, quando uma variável é usada à direita de **to** ou ←, o valor da variável é acessado. Quando uma variável é usada após **Set** ou à esquerda de ←, um valor é armazenado na variável.

O valor armazenado em uma variável pode ser um único valor (como em **0**) ou uma expressão composta por variáveis e operadores (como em **sum + num**).

Entrada/Saída

A maioria dos programas de computador simplesmente processa dados de algum tipo, portanto, devemos ser capazes de inserir valores de dados do mundo externo e exibir o resultado na tela. Podemos usar a palavra write para saída e a palavra read para entrada.

```
Write "Digite o número de valores a serem lidos e somados"
Read num
```

Os caracteres entre aspas duplas são chamados de strings e informam ao usuário o que inserir ou descrever o que está escrito. Não importa quais palavras exatas você use: **Display** ou **Print** seriam equivalentes a **Write**; **Get** ou **Input** seriam sinônimos de **Read**. Lembre-se: algoritmos de pseudocódigo são escritos para um humano traduzir para alguma linguagem de programação posteriormente. Ser consistente dentro de um projeto é um estilo melhor – tanto para você enquanto trabalha quanto para a pessoa que o segue, que pode ter que interpretar o que você escreveu.

As duas últimas instruções de saída demonstram um ponto importante:

```
Write "Err"
Write sum
```

A primeira escreve os caracteres entre aspas duplas na tela. A segunda escreve o conteúdo da variável **sum** na tela. O valor em **sum** não é alterado.

Seleção

A construção de seleção permite a escolha entre executar uma ação ou ignorá-la. A seleção também permite a escolha entre duas ações. A condição entre parênteses determina qual curso de ação seguir. Por exemplo, o seguinte segmento de pseudocódigo imprime a soma ou uma mensagem de erro. (Parece familiar?)

```
// Read and sum three numbers
if (sum < 0)
   Print error message
else
   Print sum
// Stop or whatever comes next
```

Usamos recuo para agrupar instruções (apenas uma neste caso). O controle retorna à instrução que não está recuada. O **//** apresenta um comentário ao leitor, que não faz parte do algoritmo.

Esta versão da construção de seleção é chamada de versão **if-then-else** porque o algoritmo escolhe entre duas ações. A versão **if-then** é o caso em que uma ação é executada ou ignorada. Se quiséssemos imprimir a soma em qualquer evento, poderíamos mostrar o algoritmo desta forma.

```
// Read and sum three numbers
if (sum < 0)
   Print error message
Print sum
// Stop or whatever comes next
```

Repetição

A construção de repetição permite que as instruções sejam repetidas. No problema de soma, por exemplo, um contador é inicializado, testado e incrementado. O pseudocódigo nos permite delinear o algoritmo para que o padrão fique claro. Assim como a construção de seleção, a expressão entre parênteses ao lado do **while** é um teste. Se o teste for verdadeiro, o código recuado é executado. Se o teste for falso, a execução salta para a próxima instrução não recuada.

```
Set limit to number of values to sum
while (counter < limit)
   Read num
   Set sum to sum + num
   Set counter to counter + 1
// Resto f program
```

A expressão entre parênteses ao lado de **while** e **if** é uma expressão booleana, que pode ser avaliada como verdadeira ou falsa. No **if**, se a expressão for verdadeira, o bloco recuado é executado. Se a expressão for falsa, o bloco recuado é ignorado e o bloco abaixo de **else** é executado, se existir. No **while**, se a expressão for verdadeira, o código recuado é executado. Se a expressão for falsa, a execução pula para a

próxima instrução não recuada. Estamos colocando **while**, **if** e **else** em negrito porque são frequentemente usados diretamente em várias linguagens de programação. Eles têm significados especiais em computação.

* Expressão booleana: Uma expressão que, quando avaliada, é verdadeira ou falsa.

A TABELA 6.1 resume essas instruções e mostra exemplos ou as palavras que cada uma usa. Aqui está o algoritmo de pseudocódigo para o programa que leu e somou três valores e exibiu uma mensagem de erro se o total fosse negativo:

TABELA 6.1 Instruções em Pseudocódigo.

Construtor	O que ele significa	Palavras usadas ou Exemplo
Variáveis	Representam locais nomeados nos quais valores são armazenados e de onde valores são recuperados.	Nomes que representam o papel de um valor em um problema são simplesmente escritos em pseudocódigo.
Atribuição	Armazenar um valor dentro de uma variável.	**set num to 1** **num ← 1**
Entrada/saída	Entrada: ler um valor, provavelmente do teclado. Saída: exibir o conteúdo de uma variável ou uma string, provavelmente na tela.	**read num** **get num** **write num** **display num** **write "Tenha um bom dia"**
Repetição (iteração, loop)	Repete uma ou mais instruções enquanto uma condição for avaliada como verdadeira.	**while (condição)** ** //Executa instruções recuadas**
Seleção (if-then)	Se a condição do **if** for avaliada como **true**, executa as instruções recuadas no **if**; Se a condição não for **true** (i.e., **false**), pule as instruções recuadas.	**if (novaBase = 10)** ** write "Você está convertendo"** ** write "para a mesma base".** **//Resto do código**
Seleção (if-then-else)	Se a condição do **if** for avaliada como **true**, execute as instruções recuadas no **if**; Se a condição não for **true** (i.e., **false**) execute as instruções recuadas abaixo do **else.**	**if (valor < 10)** ** write "O valor é menor do que 10"** **else** ** write "O valor é maior do que 10,"** ** write "ou é igual".** **//Resto do código**

O Pseudocódigo 1 apresenta um algoritmo para o programa que leu e somou três valores e exibiu uma mensagem de erro se o total fosse negativo.

O Pseudocódigo 2 apresenta um algoritmo para o programa que inseriu o número de valores a ler, leu-os e exibiu a soma.

Uma descrição de pseudocódigo deve eventualmente ser traduzida para um programa que possa ser executado em um computador. Uma instrução de pseudocódigo pode ter que ser traduzida para muitas instruções em linguagem assembly mas para apenas uma instrução em uma linguagem de alto nível. Por exemplo, volte e observe o último programa Pep/8. A seguir, as instruções necessárias para criar o loop no assembly do Pep/8.

```
limit:          .BLOCK 2   ; Set up a block for limit
counter:        .BLOCK 2   ; Set up counter
...
loop:
...
         LDA    counter, d ; Load limit into A register
         ADDA   1, i       ; Add one to counter
         STA    counter, d ; Store counter
         CPA    limit, d   ; Compare counter and limit
         BREQ   quit       ; Go to quit if equal
         BR     loop       ; Repeat loop
```

Pseudocódigo 1

```
// Pseudocódigo 1
set sum to 0
read num1
set sum to sum + num1
read num2
set sum to sum + num2
read num3
set sum to sum + num3
if (sum < 0)
   write 'E'
else
   write sum
```

Pseudocódigo 2

```
counter ← 0
sum ← 0
read limit
while (counter < limit)
   read num
   sum ← sum + num
   counter ← counter + 1
print sum
```

Na maioria das linguagens de alto nível, um loop pode ser escrito em uma única instrução. Nas próximas seções, exploraremos mais sobre pseudocódigo.

Seguindo um Algoritmo de Pseudocódigo

No Capítulo 2, apresentamos um algoritmo para conversão da base 10 para outras bases. Expressamos esse algoritmo em pseudocódigo para que um humano possa acompanhar.

```
while (o quociente é diferente de 0)
   dividir o número decimal pela nova base
   colocar o próximo dígito à esquerda na resposta como resto
   colocar o número decimal no quociente
```

Para refrescar a memória, aplicamos esse algoritmo para converter o número decimal 93 para octal. Dividimos 93 por 8 (a nova base), resultando em um quociente 11 e um resto 5. Esta é a primeira divisão, então 5 se torna o dígito na posição das unidades da resposta. O número decimal original (93) é substituído pelo quociente 11. O quociente não é 0, então dividimos 11 por 8, resultando em um quociente 1 e um resto 3. O dígito 3 se torna o dígito à esquerda de 5, resultando em uma resposta temporária de 35. O número decimal atual (11) é substituído pelo quociente 1. O quociente não é 0, então o dividimos por 8, resultando em um quociente 0 e um resto 1. O dígito 1 se torna o dígito mais à esquerda na resposta, resultando em um valor de 135. O quociente é 0, então o processo termina.

Este parágrafo mostra também como as descrições em pseudocódigo ainda podem ser confusas! Após a brincadeira, vamos resumir os cálculos.

Divisão	Quociente	Resto	Resposta
93/8	11	5	5
11/8	1	3	35
1/8	0	1	135

Agora, vamos começar de novo, dando nomes aos valores que precisamos manter: **numeroDecimal**, **novaBase**, **quociente**, **resto** e **resposta**. Representamos esses itens como caixas nomeadas nas quais escrevemos um valor. Veja a FIGURA 6.6(a). Colocamos um ponto de interrogação nas caixas cujo conteúdo não conhecemos. Seguindo um algoritmo, desenhamos caixas para variáveis e preenchemos os valores. O algoritmo começa perguntando se o valor em **quociente** é 0.



FIGURA 6.6 Passo a passo de um algoritmo de conversão.

? O que é Pandora?

Pandora é uma empresa de streaming de música que adapta a música tocada às preferências de cada ouvinte. A Pandora usa os resultados do Projeto Genoma Musical para fazer seleções que o ouvinte provavelmente apreciará. Em abril de 2013, o serviço de streaming de rádio da Pandora ultrapassou 200 milhões de usuários. Você ouve Pandora?[4]

Vamos supor que não, mas voltaremos a esse ponto mais tarde. A FIGURA 6.6(b) mostra os resultados após a primeira repetição do loop, dividindo 93 por 8. O quociente é 11, então repetimos o processo.

A FIGURA 6.6(c) mostra os valores após essa repetição. O quociente não é 0, então dividimos 1 por 8, resultando na situação da FIGURA 6.6(d). Agora o quociente é 0, então o processo para.

Uma de nossas caixas, **numeroDecimal**, continha originalmente o valor inicial dos dados para o problema, o número a ser convertido. Em um algoritmo de computador, devemos dar instruções a alguém no teclado para inserir esse valor. A caixa **novaBase** não mudou durante o processo, mas também deve ser inserida pelo teclado porque o algoritmo deve transformar um número decimal em alguma outra base. Portanto, a nova base – base 8 neste caso – deve ser inserida no problema.

Quando analisamos esse algoritmo, sabíamos que o **quociente** ainda não havia sido calculado então podíamos assumir que não era 0. Em um algoritmo para um computador seguir, devemos garantir que o quociente não seja 0, definindo-o inicialmente para um valor diferente de zero.

Apresentamos ao lado, o algoritmo reescrito em etapas concretas a partir das quais um programa poderia ser escrito. **DIV** é um operador que retorna o quociente decimal e **REM** é um operador que retorna o resto da divisão inteira decimal.

```
write "Informe a nova base: "
read novaBase
write "Informe o número a ser convertido: "
read numeroDecimal
resposta ← 0
quociente ← numeroDecimal
while (quociente ≠ 0)
   quociente ← numeroDecimal DIV novaBase
   resto ← numeroDecimal REM novaBase
   Coloque o resto a esquerda de resposta
   numeroDecimal ← quociente
write "A resposta é ", resposta
```

Escrevendo um Algoritmo de Pseudocódigo

Aqui, mostraremos o processo de desenvolvimento do algoritmo em pequena escala, apontando as estratégias que estamos utilizando. No Capítulo 7, abordaremos a escrita de algoritmos com mais detalhes.

Vamos ler pares de números positivos e imprimir cada par em ordem. Se houver mais de um par de números, precisamos ter um laço. Segue ao lado, uma primeira proposta do algoritmo:

```
enquanto não concluído
   imprima "Informe dois números: "
   leia o numero1
   leia o numero2
   imprima-os em ordem
```

No algoritmo anterior, como sabemos quando parar? Ou seja, como decompor "**não concluído**" em uma pergunta? Resposta: Podemos pedir ao usuário para informar quantos pares devem ser inseridos. Ao lado a segunda proposta:

```
write "Informe a quantidade de pares: "
read numeroDePares
paresLidos ← 0
while (paresLidos < numeroDePares)
   write "Informe dois números: "
   read numero1
   read numero2
   print numero1 e numero2 em ordem
```

No algoritmo anterior, como determinamos a ordem dos números? Comparamos os valores usando a construção condicional. Se **numero1** for menor que **numero2**, imprimimos **numero1** e depois **numero2**. Caso contrário, imprimimos **numero2** e depois **numero1**. Antes de completar o algoritmo, esquecemos de alguma coisa? **paresLidos** nunca muda! Devemos incrementar **paresLidos**.

Ao lado uma terceira proposta para o algoritmo:

```
write "Informe a quantidade de pares: "
read numeroDePares
paresLidos ← 0
while (paresLidos < numeroDePares)
   write "Informe dois números: "
   read numero1
   read numero2
   if (numero1 < numero2)
      print numero1, " ", numero2
   else
      print numero2, " ", numero1
   paresLidos ← paresLidos + 1
```

Ao longo do processo de escrita deste algoritmo, usamos duas estratégias principais. Fizemos perguntas e adiamos detalhes. Fazer perguntas é uma estratégia com a qual a maioria de nós está familiarizada. Adiar detalhes significa dar um nome a uma tarefa e, em seguida, preencher os detalhes de como realizá-la posteriormente. Ou seja, primeiro escrevemos o algoritmo usando **mais pares** e os **imprimimos em ordem**; em seguida, preenchemos os detalhes de como realizar essas tarefas posteriormente. Essa estratégia é conhecida como dividir para conquistar.

Um algoritmo não está completo até que seja testado. Podemos usar a mesma técnica em que nos baseamos para simular o algoritmo de conversão base: podemos escolher valores de dados e trabalhar no código com papel e lápis. Este algoritmo tem quatro variáveis que devemos rastrear: **numeroDePares**, **paresLidos**, **numero1** e **numero2**. Vamos supor que o usuário insira os seguintes dados quando solicitado:

```
3
10 20
20 10
10 10
```

A FIGURA 6.7(a) mostra os valores das variáveis no início do loop. O valor de **paresLidos** é menor que **numeroDePares**, então o loop é iniciado. O prompt é emitido e os dois números são lidos. O valor de **numero1** é 10 e **numero2** é 20, então a instrução **if** é avaliada com **true** e executa sua instrução recuada. O valor de **numero1** é impresso, seguido por **numero2**. A execução salta para fora do comando **if-else** e então o valor de **paresLidos** é incrementado. A FIGURA 6.7(b) mostra os valores no final da primeira repetição. O valor de **paresLidos** ainda é menor que **numeroDePares**, então o código é repetido. Os números são solicitados e lidos. O valor de **numero1** é 20 e **numero2** é 10, então a condição do comando **if** é avaliado como **false** e a execução salta para a instrução recuada em **else**. O valor de **numero2** é impresso, seguido por **numero1**. Em seguida, o valor de **paresLidos** é incrementado, resultando no estado das variáveis no final da segunda iteração, conforme mostrado na FIGURA 6.7(c).

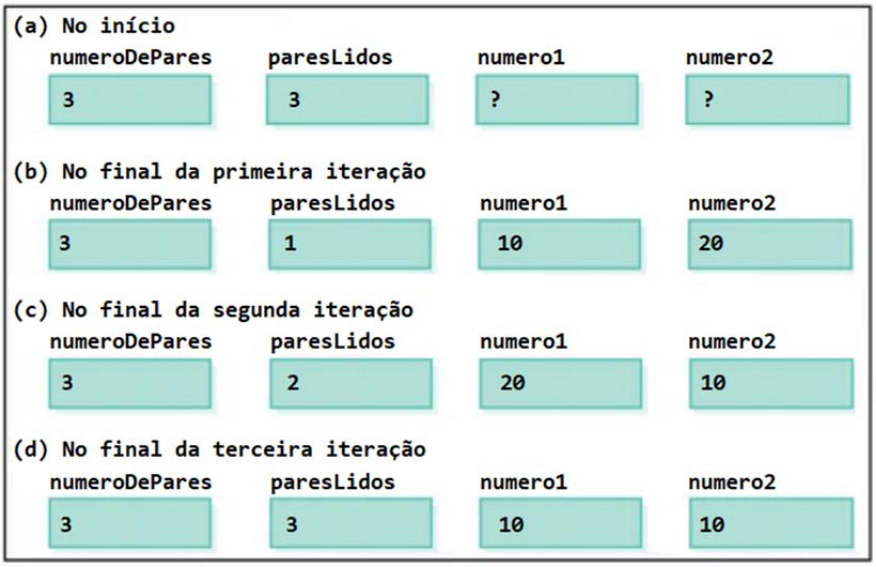


FIGURA 6.7 Passo a passo do algoritmo de pares de número.

O valor de **paresLidos** é menor que **numeroDePares**, então o código é repetido. As entradas são solicitadas e lidas, resultando em **numero1** igual a 10 e **numero2** igual a 10. Como o valor de **numero1** não é menor que **numero2**, a expressão em **if** é avaliada como **false** e a execução desvia para o comando recuado de **else**. Assim, o valor de **numero2** é impresso, seguido por **numero1**. Como os valores são os mesmos, não importa a ordem em que são impressos. O valor de **paresLidos** é incrementado. E na volta para o loop o valor de **paresLidos** não é menor que **numeroDePares**, então o código não é repetido.

Nesse processo, descrito na FIGURA 6.7, chamado de verificação (ou teste) de mesa, sentamos em uma mesa com lápis e papel e trabalhamos no projeto. É útil pegar valores de dados reais e rastrear o que acontece com eles enquanto raciocinamos sobre o projeto. Essa técnica é simples, mas incrivelmente eficaz.

* Verificação (ou teste) de mesa: Processo de rastrear a execução de um projeto (ou algoritmo) no papel.

Traduzindo um Algoritmo de Pseudocódigo

No Capítulo 1, descrevemos as camadas de linguagens que foram produzidas ao longo do tempo. Neste capítulo, começamos com a linguagem de máquina (a camada mais baixa) e avançamos um passo para a linguagem assembly. A forma como traduzimos um algoritmo de pseudocódigo depende da linguagem para

a qual estamos traduzindo o algoritmo. Aqui, onde somos limitados pelos limites de uma linguagem assembly, uma instrução de pseudocódigo requer várias instruções do Pep/8.

Escrevemos nosso algoritmo como um programa interativo. Ou seja, o programa solicita ao usuário que faça algo. Nesse caso, a primeira instrução foi escrever uma solicitação ao usuário para inserir o número de pares. Isso é excepcionalmente fácil em uma linguagem de alto nível, mas mais complicado no Pep/8. Primeiro, temos que configurar a mensagem usando uma pseudo operação .ASCII e, em seguida, configurar o código para que ele seja escrito. Vamos encurtar a mensagem para "Digite o número." A instrução STRO, que ainda não tínhamos visto antes, é usada para imprimir a mensagem.

```
mesg1:      .ASCII    "Informe número\x00"    ; First message
. . .
            STRO      mesg1                   ; Write message
```

A leitura de **numeroDePares** pode ser feita em uma única instrução Pep/8. Definir o valor de **paresLidos** como 0 pode ser feito em uma pseudo operação. Configuramos o laço carregando o valor de **paresLidos** no registrador A e comparando-o com o número a ser lido. Uma vez dentro do laço, uma segunda instrução é dada ao usuário. Vamos juntar essas peças.

```
            BR        main
mesg1:      .ASCII    "Informe número\x00"    ;
mesg2:      .ASCII    "Informe pares\x00"     ;

pairsRead:  .WORD     0x00                    ;
numPairs:   .BLOCK    2                       ;
number1:    .BLOCK    2                       ;
number2:    .BLOCK    2                       ;
Main:       STRO      mesg1, d                ;
            DECI      numPairs, d             ;
Begin:      STRO      mesg2, d                ;
            . . .

            BR        Begin                   ;
```

Agora, precisamos traduzir o corpo do loop, o que requer escrever uma mensagem, ler dois valores e compará-los. As duas primeiras tarefas exigem apenas uma instrução Pep/8 cada, mas a instrução **if-then-else** exigirá mais trabalho. Precisamos escrever o código para imprimir cada uma das duas instruções, dar nomes à primeira instrução em cada bloco de código e, em seguida, determinar qual bloco deve ser executado.

Como você pode nunca mais ver a linguagem assembly, apresentamos apenas a listagem do código-fonte. Observe que usamos um comando que não abordamos: CPA, que compara o valor no acumulador com o valor armazenado em um local na memória.

Source Code

```
                Br      Main            ; Branch to beginning of program
mesg1:          .ASCII  "Enter number\x00"  ; Set up memory
mesg2:          .ASCII  "Enter pairs\x00"   ;
numRead:        .WORD   0x00                ;
numPairs:       .BLOCK  2                   ;
number1:        .BLOCK  2                   ;
number2:        .BLOCK  2                   ;
Main:           STRO    mesg1,d     ; Prompt for number of pairs
                DECI    numPairs,d; Read number of pairs
Begin:          STRO    mesg2,d     ; Prompt for pairs
                DECI    number1,d ;
                DECI    number2,d ;
                LDA     number1,d ; Load first number
                CPA     number2,d ; Compare to second number
                BRLE    less      ; Branch to less if first less
                BR      greater   ; Branch to greater if second less
test:           LDA     numRead,d ; Increment counter
                ADDA    1,i       ;
                STA     numRead,d ;
                CPA     numPairs,d; Compare with number to be read
                BRLT    Begin     ; Repeat loop
                BR      End       ; Branch to end
less:           DECO    number1,d ; Output in order read
                DECO    number2,d ;
                BR      test      ; Repeat loop
greater:        DECO    number2,d ; Output in reverse order
                DECO    number1,d ;
                BR      test      ; Repeat loop
End:            STOP              ; Stop execution
                .END              ; Stop assembly
```

Konrad Zuse



© Karl Staedele/dpa/Corbis

Konrad Zuse nasceu em Berlim em 22 de junho de 1910. Frequentou o Ginásio Humanístico e estudou engenharia civil na Universidade Técnica de Berlim-Charlottenburg. Após a formatura, trabalhou brevemente para a Ford Motor Company e, em seguida, para a Henschel & Son, fabricante de equipamentos de transporte. Em 1935, no apartamento de seus pais, Zuse começou a fazer experiências na construção de uma máquina para realizar os cálculos tediosos exigidos na engenharia. Sua primeira tentativa foi a Z1, que possuía uma calculadora mecânica binária de ponto flutuante. O pedido de patente de Zuse em 1936 antecipou a arquitetura de von Neumann em nove anos, com programa e dados modificáveis no armazenamento.[7] Zuse serviu brevemente na infantaria alemã, mas convenceu o exército a deixá-lo retornar à construção de computadores.[8]

Entre 1936 e 1945, Zuse continuou a aprimorar o Z1 com suas máquinas Z2, Z3 e Z4. O Z1, juntamente com seus projetos originais, foi destruído em um ataque aéreo britânico na Segunda Guerra Mundial. Devido à Segunda Guerra Mundial, Zuse trabalhou em isolamento quase total, sem saber nada sobre o trabalho de Turing, Atanasoff ou Eckert e Mauchly.

Em 1946, Zuse fundou uma das primeiras empresas de computadores e, em 1949, fundou uma segunda empresa. A Zuse KG produziu o Z4, o primeiro computador comercial do mundo, e o entregou à ETH Zurique. A Zuse KG construiu um total de 251 computadores e foi vendida para a Siemens. Enquanto trabalhava no Z4, Zuse concluiu que programar em código de máquina era muito complicado. Com o avanço da guerra, ele teve que fugir de Berlim para o interior, onde não havia hardware para trabalhar. Lá, ele projetou a primeira linguagem de programação de alto nível, Plankalkul (Plano de Cálculo). Embora nunca tenha sido implementada, esta foi a primeira tentativa de uma linguagem de programação geral. No final da década de 1980, com financiamento da Siemens e de outras cinco empresas de computadores, Zuse construiu uma reconstrução funcional do Z1, que agora está em exposição no Deutsches Technikmuseum em Berlim. Konrad Zuse casou-se com Gisela Brandes em janeiro de 1945, em uma cerimônia bastante formal.

O casal teve cinco filhos. Ele faleceu em 18 de dezembro de 1995, na Alemanha. O centenário de seu nascimento foi comemorado com exposições, palestras e workshops para relembrar a vida e o legado deste pioneiro da computação.

? Certificação e licenciamento

Certificação é o processo pelo qual organizações não governamentais concedem reconhecimento a um indivíduo que atendeu a qualificações predeterminadas especificadas pela organização. O licenciamento é uma concessão governamental de autoridade legal, que não é voluntária. Quase qualquer pessoa que presta um serviço ao público precisa ser licenciada ou certificada; profissionais de informática, não.

6.6 Testes

Testamos brevemente nossos programas, executando-os para verificar se produziam a saída esperada. No entanto, testar envolve muito mais do que apenas executar o programa uma vez. Vamos analisar os testes em mais detalhes no programa que lê três números e imprime sua soma. Como testamos um programa específico para determinar sua correção? Projetamos e implementamos um plano de teste. Um plano de teste é um documento que especifica quantas vezes e com quais dados um programa deve ser executado para testá-lo completamente. Cada conjunto de valores de dados de entrada é chamado de caso de teste. O plano de teste deve listar o motivo da escolha dos dados, os valores dos dados e a saída esperada de cada caso.

* Plano de teste: Um documento que especifica como um programa deve ser testado.

Os casos de teste devem ser escolhidos cuidadosamente. Diversas abordagens de teste podem ser usadas para guiar esse processo. Cobertura de código é uma abordagem que projeta casos de teste para garantir que cada instrução no programa seja executada. Como o código é visível para o testador, essa abordagem também é chamada de teste de caixa branca (ou ainda teste de caixa clara ou teste estrutural).

A cobertura de dados, outra abordagem, exige o design de casos de teste para garantir que os limites dos dados permitidos sejam cobertos. Como essa abordagem se baseia apenas nos dados de entrada e não no código, ela também é chamada de teste de caixa-preta. Frequentemente, a fase de testagem envolve uma combinação dessas duas abordagens.

* Teste de cobertura de código (caixa-branca): Testar um programa ou subprograma com base na cobertura de todas as instruções do código.

* Teste de cobertura de dados (caixa-preta): Testar um programa ou subprograma com base nos possíveis valores de entrada, tratando o código como uma caixa-preta.

A implementação do plano de teste envolve a execução de cada um dos casos de teste descritos no plano de teste e o registro dos resultados. Se os resultados não forem os esperados, você deve retornar ao seu projeto e encontrar e corrigir o(s) erro(s). O processo para quando cada um dos casos de teste fornece os resultados esperados. Observe que um plano de teste implementado nos dá uma medida de confiança de que o programa está correto; no entanto, tudo o que sabemos com certeza é que nosso programa funciona corretamente nos casos de teste. Portanto, a qualidade dos casos de teste é extremamente importante.

* Implementação do plano de teste: Usar os casos de teste especificados em um plano de teste para verificar se um programa gera os resultados previstos.

No caso do programa que lê três valores e os soma, um teste de caixa-branca incluiria apenas três valores de dados. Não há instruções condicionais neste programa para testar com dados alternativos. No entanto, um teste de caixa-branca não seria suficiente aqui, porque precisamos testar valores de dados negativos e positivos. Os números que estão sendo lidos são armazenados em uma palavra. O problema não limita os valores a $\pm 2^{15} - 1$, mas nossa implementação limita. Também devemos tentar valores nos limites do tamanho da entrada permitida no plano de teste, mas, como eles estão sendo somados, precisamos ter certeza de que a soma não exceda $\pm 2^{15} - 1$.

Razão para o caso de teste	Valores de entrada	Saída esperada	Saída observada
Suposição: os valores de entrada não são maiores que 2^{15} nem menores que -2^{15}.			
Insira três números positivos.	4, 6, 1	11	11
Insira três números negativos.	-4, -6, -1	-11	-11
Insira números mistos.	4, 6, -1	9	9
	4, -6, -1	-1	-1
	-4, 6, 1	3	3
Insira números grandes.	32767, -1, +1	32767	32767

Para implementar este plano de teste, executamos o programa seis vezes, uma para cada caso de teste. Os resultados foram então escritos na coluna "Saída observada". Eles foram o que havíamos previsto.

RESUMO

Um computador pode armazenar, recuperar e processar dados. Um usuário pode inserir dados na máquina, e a máquina pode exibir os dados para que o usuário possa vê-los. No nível mais baixo de abstração, as instruções para a máquina estão diretamente relacionadas a essas cinco operações.

A linguagem de máquina de um computador é o conjunto de instruções que o hardware da máquina foi construído para reconhecer e executar. Programas em linguagem de máquina são escritos inserindo uma série dessas instruções em sua forma binária. O Pep/8 é um computador virtual com um registrador A e instruções de duas partes. Uma parte da instrução informa qual ação a instrução executa e a outra parte especifica onde os dados a serem usados (se houver) podem ser encontrados. Programas escritos usando o conjunto de instruções Pep/8 podem ser executados usando um simulador – um programa que se comporta como o computador Pep/8.

A linguagem assembly do Pep/8 é uma linguagem que permite ao usuário inserir códigos mnemônicos para cada instrução, em vez de números binários. Programas escritos em linguagem assembly são traduzidos para seus equivalentes em linguagem de máquina, que são então executados usando o simulador Pep/8.

Pseudocódigo é uma linguagem abreviada que as pessoas usam para expressar algoritmos. Ela permite ao usuário nomear variáveis (locais para inserir valores), inserir valores em variáveis e imprimir os valores

armazenados em variáveis. O pseudocódigo também nos permite descrever algoritmos que repetem ações e escolhem entre ações alternativas. Fazer perguntas e adiar detalhes são duas estratégias de resolução de problemas usadas no projeto de algoritmos.

Programas, assim como algoritmos, devem ser testados. O teste de cobertura de código envolve determinar a entrada do programa examinando cuidadosamente o código do programa. O teste de cobertura de dados envolve determinar a entrada considerando todos os valores de entrada possíveis.

EXERCÍCIOS

Para os Exercícios 1 a 15, marque as respostas como V para verdadeiras ou F para falsas:

1. A aritmética pode ser executada no registrador de instruções.
2. A aritmética pode ser executada no registrador A.
3. A aritmética pode ser executada no acumulador.
4. **LDA 0X008B, i** carrega 008B no registrador A.
5. **ADDA 0x008B, i** adiciona o conteúdo de 008B ao registrador A.
6. O contador de programa e o registrador de instruções são dois nomes para o mesmo local.
7. O registrador A e o acumulador são dois nomes para o mesmo local.
8. O registrador de instruções tem 3 bytes de comprimento.
9. O contador de programa tem 3 bytes de comprimento.
10. A instrução de desvio, BR, desvia para o local especificado no especificador de operando.
11. O especificador de instrução tem 1 byte de comprimento.
12. Se os dados a serem carregados no acumulador estiverem armazenados no operando, o especificador de instrução será 00.
13. Se os dados no acumulador forem armazenados no local indicado no operando, o especificador de instrução será 00.
14. Todas as instruções Pep/8 ocupam 3 bytes.
15. Pelo menos uma instrução de ramificação é necessária em um loop.

Dado o estado de memória (em hexadecimal) ao lado, complete os Exercícios 16 a 20 associando o problema à solução apresentada.

a) A2 11 b) A2 12 c) 00 02 d) 11 00 e) 00 FF

```
0001 A2
0002 11
0003 00
0004 FF
```

16. Qual é o conteúdo do registrador A após a execução desta instrução?
    C1 00 01
17. Qual é o conteúdo do registrador A após a execução desta instrução?
    C1 00 02
18. Qual é o conteúdo do registrador A após a execução das duas instruções a seguir?
    C0 00 01 70 00 01
19. Qual é o conteúdo do registrador A após a execução das duas instruções a seguir?
    C1 00 01 70 00 01
20. Qual é o conteúdo da posição 0001 após a execução das duas instruções a seguir?
    C1 00 03 **E0 00 01**

Os exercícios 21 a 60 são programas ou perguntas de resposta curta.

21. O que significa dizer que um computador é um dispositivo programável?
22. Liste cinco operações que qualquer linguagem de máquina deve incluir.
23. Quantas tarefas de baixo nível cada instrução em linguagem de máquina pode executar?
24. O que é uma máquina virtual? Discuta esta definição em termos do computador Pep/8.

25. Quantos bits uma instrução ocupa no Pep/8?
26. Descreva os recursos da CPU Pep/8 que abordamos neste capítulo.
27. Onde estão os dados (operando) se o especificador do modo de endereço for 000? E se for 001?
28. Discutimos dois especificadores de modo. Quantos existem?
29. Distinga entre o IR (registrador de instruções) e o PC (contador de programa).
30. Quantos bits são necessários para endereçar a memória do Pep/8?
31. Quantas células a mais poderiam ser adicionadas à memória sem precisar alterar o formato da instrução? Justifique sua resposta.
32. Algumas instruções do Pep/8 são unárias, ocupando apenas 1 byte. Outras instruções requerem 3 bytes. Dadas as instruções que abordamos neste capítulo, seria útil definir instruções que requerem apenas 2 bytes?
33. Se o caractere de entrada for A, qual é o resultado da execução das duas instruções a seguir?

```
0001 49 00 06                    0004 51 00 06
```

34. Se o número de entrada for 5, qual é o conteúdo do registrador A após a execução das instruções a seguir?

```
0001 31 00 0F          0004 C1 00 0F          0007 70 00 02
```

35. Escreva o algoritmo para escrever seu nome, considerando que a linguagem de implementação é o código de máquina do Pep/8.
36. Escreva o programa em linguagem de máquina para implementar o algoritmo do Exercício 35.
37. Escreva o algoritmo para escrever seu nome, considerando que a linguagem de implementação é a linguagem assembly do Pep/8.
38. Escreva o programa em linguagem assembly para implementar o algoritmo do Exercício 37.
39. Reescreva o programa de exemplo da Seção 6.3 usando endereçamento direto.
40. Diferencie as opções de menu do Pep/8: Assemble, Load e Execute (run).
41. O programa a seguir parece rodar, mas apresenta comportamentos estranhos com certos valores de entrada. Você consegue encontrar o erro?

```
        BR      Main
sum:    .WORD   0x0000
num1:   .BLOCK  1
num2:   .BLOCK  1
num3:   .BLOCK  1
Main:   LDA     sum, d
        DECI    num1, d
        DECI    num2, d
        DECI    num3, d
        ADDA    num3, d
        ADDA    num2, d
        ADDA    num1, d
        STA     sum, d
        DECO    sum, d
        STOP
        .END
```

42. Corrija o código do Exercício 41 e execute o plano de teste descrito no capítulo.
43. Conclua a execução do plano de teste para o algoritmo do texto que lê e soma três valores.
44. Escreva um algoritmo em pseudocódigo que leia três valores e escreva o resultado da subtração do segundo valor da soma do primeiro e do terceiro valores.
45. Implemente o algoritmo do Exercício 44 como um programa em linguagem assembly.
46. Escreva e implemente um plano de teste para o programa do Exercício 45.
47. Projete e implemente, em linguagem assembly, um algoritmo que leia quatro valores e imprima a soma.
48. O plano de teste para um programa em linguagem de máquina é válido para a mesma solução escrita em linguagem assembly? Explique sua resposta.
49. Distinga entre as pseudo operações .BLOCK e .WORD.

50. Distinguir entre pseudo operações em linguagem assembly e instruções mnemônicas.
51. Distinguir entre planos de teste baseados em cobertura de código e cobertura de dados.
52. Qual botão no console do Pep/8 deve ser clicado para entrada de dados pelo teclado?
53. Escreva a instrução em linguagem assembly do Pep/8 para as seguintes instruções:
    a) Desviar para o local Branch1 se o acumulador for zero.
    b) Desviar para o local Branch1 se o acumulador for negativo.
    c) Desviar para o local Branch1 se o acumulador for negativo e para Branch2 se o acumulador não for negativo.
54. Escreva um algoritmo de pseudocódigo para ler um nome e escrever uma mensagem de "Bom dia".
55. Escreva um algoritmo de pseudocódigo para obter três inteiros do usuário e imprimi-los em ordem numérica crescente.
56. Inclua o projeto do Exercício 55 em um laço que lê os três valores até que o usuário insira o primeiro valor do trio como negativo.
57. Reescreva o algoritmo do Exercício 56 de forma que o usuário precise inserir apenas um valor negativo para parar (ou seja, o segundo e o terceiro valores não são inseridos).
58. Distinga entre pseudocódigo e pseudo operações.
59. Quais são as construções que o pseudocódigo deve ser capaz de expressar?
60. Distinga entre a construção de laço e a construção de seleção.

QUESTÕES PARA REFLEXÃO

1. Você gostaria de programar em linguagem assembly? Você consegue pensar em algum tipo de personalidade que seria adequado para esse trabalho detalhado?
2. O processo de tradução foi demonstrado mostrando o programa em linguagem de máquina resultado do programa em linguagem assembly. Observe atentamente a solução do Exercício 45. Pense nas etapas que o programa montador deve executar. Você acha que a tradução pode ser feita examinando cada instrução em linguagem assembly uma vez ou cada uma deve ser examinada duas vezes? Convença um amigo de que você está certo.
3. Se uma pessoa possui dois computadores do mesmo tipo, é ético comprar uma cópia de um pacote de software e instalá-lo em ambas as máquinas? Quais são os argumentos a favor do sim? Quais são os argumentos a favor do não?
4. Alguém já pegou software emprestado com você? Você acha que ele ou ela o copiou?